EDUGENIE: GOOGLE GEMINI POWERED LEARNING ASSISTANT

PROJECT DOCUMENTATION

1. PROJECT DESCRIPTION

EduGenie is a lightweight AI-powered educational assistant designed to help students learn more easily using Generative AI.

The application uses **Google Gemini** as the AI engine and **FastAPI** as the backend framework. A simple HTML, CSS and JavaScript frontend provides an easy-to-use interface for students.

EduGenie provides the following major educational features:

- Question and Answer
- Simplified Topic Explanation
- Quiz Generation
- Text Summarization
- Learning Path Recommendations

The system accepts a student's question, topic, or educational content through the web interface. The request is sent to the FastAPI backend, which processes the request using the appropriate Python module. The selected module sends a structured prompt to Google Gemini and returns the generated result to the frontend.

The main objective of EduGenie is to provide students with a simple and interactive learning assistant that can support different stages of learning.

2. OBJECTIVES

The main objectives of the EduGenie project are:

1. To develop an AI-powered educational assistant.
2. To provide quick answers to student questions.
3. To explain difficult topics using simple language.
4. To automatically generate multiple-choice quizzes.
5. To summarize long educational content.
6. To provide structured learning paths from beginner to advanced levels.
7. To provide a simple and responsive web interface.
8. To integrate Google Gemini with a FastAPI backend.
9. To demonstrate the practical use of Generative AI in education.

3. USE CASES / SCENARIOS

Scenario 1: Question and Answer

A student wants to know:

"Which is the largest ocean?"

The student selects **Q&A**, enters the question and clicks **Generate Result**.

EduGenie sends the question to Google Gemini and displays the answer on the webpage.

Scenario 2: Topic Explanation

A student wants to understand a difficult topic such as:

"Pythagoras Theorem"

The student selects **Explain a Topic**.

EduGenie provides:

- Definition
- Key points
- Simple example
- Real-life or academic use

Scenario 3: Quiz Generation

A student provides educational content and selects **Generate Quiz**.

EduGenie generates exactly three multiple-choice questions. Each question contains four options, the correct answer and a short explanation.

Scenario 4: Summarization

A student has a long educational paragraph and wants a quick revision version.

The student selects **Summarize**.

EduGenie generates a concise summary while keeping the important information.

Scenario 5: Learning Path

A student wants to learn a topic such as Python, SQL or Java.

The student selects **Recommend Learning Path**.

EduGenie provides a structured path containing:

- Prerequisites
- Beginner Level
- Intermediate Level
- Advanced Level
- Practice Activities
- Recommended Resource Types
- Suggested Learning Order

4. TECHNOLOGIES USED

The following technologies are used in the EduGenie project:

4.1 Python

Python is used as the main programming language for the backend and AI-related functionality.

4.2 FastAPI

FastAPI is used to create the backend application and REST API endpoints.

4.3 Google Gemini API

Google Gemini provides the Generative AI capabilities used by EduGenie.

It is used for:

- Question answering
- Topic explanation
- Quiz generation
- Summarization
- Learning recommendations

4.4 HTML

HTML is used to create the structure of the web interface.

4.5 CSS

CSS is used to design and style the web interface.

4.6 JavaScript

JavaScript is used to connect the frontend with the FastAPI backend and display results dynamically.

4.7 Jinja2

Jinja2 is used as the HTML templating engine for serving the frontend through FastAPI.

4.8 Uvicorn

Uvicorn is used as the ASGI server to run the FastAPI application locally.

4.9 python-dotenv

The `python-dotenv` package is used to load configuration values such as the Gemini API key from the `.env` file.

5. PREREQUISITES

Before running EduGenie, the following requirements are needed:

5.1 Python

Python 3.10 or later should be installed.

Python installation can be verified using:

```
python --version
```

5.2 FastAPI

FastAPI is required for creating the backend API.

5.3 Google Gemini API Key

A Google Gemini API key is required for communicating with the Gemini model.

The API key should be stored securely in the `.env` file.

Example:

```
GEMINI_API_KEY=YOUR_ACTUAL_GEMINI_API_KEY
GEMINI_MODEL=gemini-3.8-flash
```

The actual API key should not be shared publicly.

5.4 Uvicorn

Uvicorn is required to run the FastAPI application.

5.5 Jinja2

Jinja2 is required for rendering the HTML frontend.

5.6 VS Code

Visual Studio Code can be used to create and manage the project files.

6. PROJECT ARCHITECTURE

The EduGenie project follows a modular architecture.

```
EduGenie/
│
├── .env
├── .gitignore
├── main.py
├── gemini_client.py
├── qna.py
├── explanation_module.py
├── quiz_module.py
├── summary_module.py
├── learning_path.py
├── requirements.txt
│
```

```
├── templates/
│   └── index.html
│
└── static/
    ├── app.js
    └── style.css
```

Description of Files

File / Folder	Purpose
`.env`	Stores Gemini API configuration
`.gitignore`	Specifies files that should not be tracked by Git
`main.py`	Main FastAPI application
`gemini_client.py`	Connects the application with Google Gemini
`qna.py`	Question-answering functionality
`explanation_module.py`	Topic explanation functionality
`quiz_module.py`	Quiz generation functionality
`summary_module.py`	Text summarization functionality
`learning_path.py`	Learning recommendation functionality
`requirements.txt`	Lists required Python packages
`templates/index.html`	Frontend HTML page
`static/style.css`	Frontend styling
`static/app.js`	Frontend JavaScript functionality

---

7. SYSTEM WORKFLOW

The overall workflow of EduGenie is:

```
Student
   ↓
EduGenie Web Interface
```

```
   ↓
Select Task
   ↓
Enter Question / Topic / Content
   ↓
JavaScript sends POST request
   ↓
FastAPI Backend
   ↓
Selected Python Module
   ↓
Gemini Client
   ↓
Google Gemini API
   ↓
AI Generated Result
   ↓
FastAPI Response
   ↓
JavaScript
   ↓
Result displayed on Webpage
```

The frontend provides the user interface, while FastAPI manages the backend requests. Each task is connected to a separate Python module.

8. MILESTONE 1: MODEL AND ARCHITECTURE SETUP

Activity 1.1: Google Gemini Integration

EduGenie uses Google Gemini as its Generative AI service.

The `gemini_client.py` file creates the Gemini client and sends prompts to the selected Gemini model.

The project uses:

```
gemini-2.5-flash
```

The API key is loaded from the `.env` file rather than being directly written into the Python source code.

This provides a cleaner and safer configuration approach.

Activity 1.2: Gemini Client

The `generate_text()` function receives a prompt and sends it to Google Gemini.

The generated response is returned as text to the individual EduGenie modules.

This common Gemini client allows all modules to use the same AI connection.

9. MILESTONE 2: CORE FUNCTIONALITIES

Activity 2.1: Q&A Module

The Q&A functionality is implemented in:

`qna.py`

The `answer_question()` function receives the student's question and creates a structured prompt.

The prompt instructs Gemini to:

- Answer accurately.
- Use simple English.
- Give a direct answer.
- Explain important points when necessary.
- Avoid inventing facts.
- Handle unclear questions carefully.

The generated answer is returned to the FastAPI backend.

Activity 2.2: Explanation Module

The explanation functionality is implemented in:

`explanation_module.py`

The `explain_topic()` function asks Gemini to explain a topic in beginner-friendly English.

The response follows this structure:

1. Definition
2. Key Points
3. Simple Example
4. Real-Life or Academic Use

This structure helps students understand unfamiliar topics more easily.

Activity 2.3: Quiz Module

The quiz functionality is implemented in:

`quiz_module.py`

EduGenie generates exactly **three multiple-choice questions**.

Each question contains:

- Question
- Four options
- Correct answer
- Short explanation

The module requests the response in JSON format.

It also validates the response to make sure:

- The response is a list.
- There are exactly three questions.
- Every question contains four options.
- The correct answer exists among the options.

The module also removes Markdown code fences if Gemini returns JSON inside a code block.

This makes the generated quiz easier to process and display on the frontend.

Activity 2.4: Summary Module

The summarization functionality is implemented in:

`summary_module.py`

The `summarize_text()` function sends the educational content to Gemini.

The prompt instructs Gemini to:

- Keep important information.
- Remove unnecessary repetition.
- Use simple English.
- Make the output useful for quick revision.
- Avoid adding information that is not present in the original content.

This feature is useful when students need to revise long paragraphs quickly.

Activity 2.5: Learning Path Module

The learning recommendation functionality is implemented in:

`learning_path.py`

The `get_learning_recommendations()` function generates a structured learning plan.

The response is organized into:

1. Prerequisites
2. Beginner Level
3. Intermediate Level
4. Advanced Level
5. Practice Activities
6. Recommended Resource Types
7. Suggested Learning Order

This helps students understand what they should learn first and how they can gradually progress to advanced topics.

10. MILESTONE 2.2: BACKEND API WITH FASTAPI

The backend is implemented in:

`main.py`

FastAPI provides separate endpoints for each major functionality.

API Endpoints

```
POST /qa
POST /explain
POST /quiz
POST /summarize
POST /learn/recommendations
```

There is also a health-check endpoint:

```
GET /health
```

The health endpoint returns:

```
{
```

```
    "status": "ok",
    "service": "EduGenie"
}
```

The backend receives the task and text from the frontend and sends the request to the appropriate module.

11. MILESTONE 3: FRONTEND DEVELOPMENT

Activity 3.1: HTML Interface

The frontend is created using:

`templates/index.html`

The webpage contains:

- EduGenie title
- Task selection dropdown
- Text area
- Generate Result button
- Clear button
- Status message
- Result section

The task dropdown provides five options:

```
Q&A
Explain a Topic
Generate Quiz
Summarize
Recommend Learning Path
```

12. CSS STYLING

The frontend design is implemented in : `static/style.css`

The CSS provides:

- Page layout
- Header design
- Cards
- Buttons
- Input styling
- Result section

- Responsive design
- Mobile-friendly layout
- The interface is designed to remain usable on different screen sizes.

13. JAVASCRIPT INTEGRATION

The frontend functionality is implemented in:

```
static/app.js
```

JavaScript connects the webpage to the FastAPI backend.

The application uses the following endpoint mapping:

```
Q&A                 → /qa
Explain             → /explain
Quiz                → /quiz
Summarize           → /summarize
Learning Path       → /learn/recommendations
```

When the user clicks **Generate Result**, JavaScript:

1. Reads the selected task.
2. Reads the entered text.
3. Selects the correct backend endpoint.
4. Sends a POST request.
5. Receives the backend response.
6. Displays the result on the webpage.

For quizzes, JavaScript separately displays the questions, options, answers and explanations.

14. MILESTONE 4: DEPLOYMENT

Activity 4.1: Install Dependencies

The project dependencies are stored in:

```
requirements.txt
```

The required packages can be installed using:

```
pip install -r requirements.txt
```

Activity 4.2: Run the Application

The FastAPI application can be started using:

```
uvicorn main:app --reload
```

After starting the server, the application can be accessed at:

```
http://127.0.0.1:8000
```

15. FUNCTIONAL TESTING

The following features should be tested after running the application.

Test 1: Q&A

Select:

```
Q&A
```

Enter a question and click:

```
Generate Result
```

Expected result:

The AI-generated answer should appear in the Result section.

Test 2: Explanation

Select:

```
Explain a Topic
```

Enter a topic such as:

```
What is Artificial Intelligence?
```

Expected result:

The application should provide a definition, key points, example and use.

Test 3: Quiz

Select:

```
Generate Quiz
```

Enter educational content.

Expected result:

Three MCQs should be generated, with four options for each question.

Test 4: Summary

Select:

```
Summarize
```

Enter a long paragraph.

Expected result:

A shorter revision-friendly summary should appear.

Test 5: Learning Path

Select:

```
Recommend Learning Path
```

Enter a topic such as:

```
Python Programming
```

Expected result:

A structured learning path from prerequisites to advanced level should be generated.

16. EXPECTED APPLICATION FEATURES

After successful implementation, EduGenie provides:

Question Answering

Students can ask educational questions and receive AI-generated answers.

Topic Explanation

Students can understand difficult concepts through simple explanations.

Quiz Generation

Students can test their understanding using automatically generated MCQs.

Summarization

Students can convert lengthy educational content into concise revision material.

Learning Recommendations

Students can receive a structured learning path for a selected topic.

17. ADVANTAGES

The EduGenie system provides several advantages:

1. Easy-to-use educational interface.
2. AI-powered question answering.
3. Simple explanations for difficult topics.
4. Automatic quiz generation.
5. Quick summarization for revision.
6. Structured learning recommendations.
7. Modular Python architecture.
8. FastAPI-based backend.
9. Responsive frontend.
10. Integration with Google Gemini.

18. LIMITATIONS

The current version has some limitations:

1. AI-generated responses may require verification.
2. The application depends on access to the Gemini API.

3. An API key is required.
4. The current application does not provide user accounts.
5. Learning progress is not permanently tracked.
6. Voice interaction is not currently implemented.
7. Image and PDF-based learning input is not currently implemented.
8. The application is primarily designed as a lightweight web application.

---

20. CONCLUSION

EduGenie is an AI-powered educational assistant developed using Python, FastAPI, Google Gemini, HTML, CSS and JavaScript. The system provides multiple learning features including question answering, topic explanation, quiz generation, summarization and learning path recommendations. The modular architecture separates each educational function into an individual Python module, making the project easier to understand, maintain and extend.Google Gemini provides the generative AI capabilities, while FastAPI manages communication between the frontend and backend. The HTML, CSS and JavaScript frontend provides a simple interface through which students can interact with the system.The project demonstrates how Generative AI can be integrated into an educational application to provide interactive and personalized learning support.Future versions of EduGenie can be enhanced with voice interaction, multilingual support, mobile applications, progress tracking, adaptive learning, gamification, image/PDF processing and Learning Management System integration. Overall, EduGenie demonstrates the practical application of Generative AI, API integration, backend development and frontend development in the education domain.

21. REFERENCES

1. Google Gemini API / Google AI for Developers
2. FastAPI Documentation
3. Python Documentation
4. Uvicorn Documentation
5. Jinja2 Documentation
6. HTML Documentation
7. CSS Documentation
8. JavaScript Documentation
9. Python-dotenv Documentation
10. Project reference document provided for EduGenie